

Git-Bash-Übung: Branches und Merge

Durchgängige Selbstlernübung am Beispiel einer Café-Website

Ausgangssituation: Die Website des Café Morgenrot ist bereits als kleines Git-Projekt angelegt. Neue Funktionen sollen jetzt parallel entwickelt, getestet und anschließend sauber zusammengeführt werden.

Lernziele

- den Sinn von Branches erklären
- Branches anlegen und zwischen ihnen wechseln
- Änderungen in getrennten Entwicklungszweigen committen
- Fast-Forward-Merge und Merge-Commit unterscheiden
- Merge-Konflikte erkennen und lösen
- den Verlauf mit `git log --oneline --graph --all` lesen

1. Warum Branches?

Neue Funktionen sollten in der Praxis nicht direkt im Hauptzweig entwickelt werden. Branches schaffen getrennte Arbeitsbereiche. Dadurch bleibt main stabil, während neue Ideen oder Korrekturen in Ruhe vorbereitet werden können.

Merksatz: Ein Branch ist ein eigener Entwicklungszweig. Er erlaubt es, Änderungen parallel und voneinander getrennt zu bearbeiten.

- neue Funktion testen, ohne die stabile Version zu gefährden
- mehrere Aufgaben parallel bearbeiten
- Arbeit verschiedener Personen sauber trennen
- Änderungen erst nach Prüfung in main übernehmen

2. Projekt vorbereiten

Lege einen Ordner `cafe-website-branches` an, öffne Git Bash in diesem Ordner und initialisiere dort das Repository.

```
git init
```

Erstelle anschließend die drei Dateien index.html, styles.css und script.js mit folgendem Inhalt.

index.html

```
<!DOCTYPE html>
<html lang="de">
<head>
  <meta charset="UTF-8">
  <title>Café Morgenrot</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>Café Morgenrot</h1>
    <p>Frischer Kaffee und hausgemachter Kuchen</p>
  </header>

  <main>
    <section>
      <h2>Willkommen</h2>
      <p>Besuchen Sie unser gemütliches Café im Stadtzentrum.</p>
    </section>
  </main>

  <script src="script.js"></script>
</body>
</html>
```

styles.css

```
body {
  font-family: Arial, sans-serif;
  margin: 20px;
  background-color: #f8f1e7;
  color: #3a2c24;
}

header {
  background-color: #c89f7b;
  padding: 20px;
}
```

script.js

```
console.log("Café Morgenrot geladen");
```

Prüfe nun den Status, füge die Dateien zur Staging Area hinzu und erstelle den ersten Commit.

```
git status
git add .
git commit -m "Startversion der Café-Website erstellt"
```

```
git log
```

Selbstkontrolle

- ☐ Welche Dateien liegen jetzt im Repository?
- ☐ Was unterscheidet Working Directory, Staging Area und Repository?
- ☐ Warum ist der erste Commit ein sinnvoller Ausgangspunkt für Branches?

3. Den aktuellen Branch verstehen

Zeige die vorhandenen Branches und die Historie an. Meist heißt der Hauptbranch main.

- 1.
- 2.

Beobachtung: Im Moment gibt es nur einen Entwicklungszweig. Alle weiteren Branches gehen von diesem Stand aus.

4. Erster Feature-Branch: Speisekarte

Das Café möchte eine neue Rubrik „Unsere Spezialitäten“. Diese Änderung soll zuerst getrennt entwickelt werden.

1. `git branch speisekarte`
2. `git switch speisekarte`

Alternative in einem Schritt:

Ergänze in index.html im main-Bereich folgenden Abschnitt:

```
<section>
  <h2>Unsere Spezialitäten</h2>
  <ul>
    <li>Espresso</li>
    <li>Cappuccino</li>
    <li>Apfelstrudel</li>
  </ul>
</section>
```

Diese Version der Website soll dauerhaft im Repository gespeichert werden.

```
git status
git diff
git add index.html
git commit -m "Spezialitäten zur Startseite hinzugefügt"
```

Selbstkontrolle

- ☐ Warum ist die Änderung noch nicht in main sichtbar?
- ☐ Was zeigt git diff vor dem Commit?
- ☐ Warum ist ein eigener Feature-Branch hier sinnvoll?

5. Zwischen Branches wechseln

Wechsle zurück nach main und öffne die index.html. Die neue Rubrik ist dort noch nicht vorhanden. Wechsle danach wieder in den Branch speisekarte.

- 1.
- 2.

Erkenntnis: Ein Branch ist kein zweiter Ordner. Git zeigt beim Wechsel denselben Projektordner in einem anderen Versionsstand.

6. Fast-Forward-Merge

Die Änderung ist fertig und soll in main übernommen werden. Da main seit der Abzweigung nicht weiter verändert wurde, ist der Merge meist besonders einfach.

- 1.
- 2.
- 3.

Begriff: Bei einem Fast-Forward-Merge wird main einfach auf den neuesten Commit des Feature-Branche vorgeschoben.

Lösche den Branch danach, weil er nicht mehr benötigt wird.

7. Parallele Entwicklung in zwei Branches

Jetzt werden zwei Aufgaben unabhängig voneinander vorbereitet: ein Design-Update und eine Sommeraktion.

Branch 1: design

```
git switch -c design
```

```
main {  
  background-color: white;  
  padding: 20px;  
  border-radius: 10px;  
}  
  
h1 {  
  color: #5c3b28;  
}
```

```
git add styles.css  
git commit -m "Design der Café-Website verbessert"
```

Branch 2: sommeraktion

```
git switch main  
git switch -c sommeraktion
```

```
<section>  
  <h2>Sommeraktion</h2>  
  <p>Eiskaffee zum Aktionspreis!</p>  
</section>
```

```
git add index.html  
git commit -m "Sommeraktion auf Startseite ergänzt"  
git log --oneline --graph --all
```

Selbstkontrolle

- ☐ Welche Änderungen liegen in design, welche in sommeraktion?
- ☐ Warum können beide Aufgaben gleichzeitig vorbereitet werden?
- ☐ Was zeigt die grafische Historie jetzt?

8. Merge mit eigenem Merge-Commit

Wenn sich main und ein Branch unabhängig weiterentwickelt haben, entsteht oft ein echter Merge-Commit.

- 1.
- 2.
- 3.
- 4.

Lösche anschließend beide Feature-Branches.

- 1.
- 2.

Beobachtung: Im Log ist jetzt sichtbar, dass zwei Entwicklungszweige wieder zusammengeführt wurden.

9. Merge-Konflikt gezielt auslösen

Ein Konflikt entsteht, wenn zwei Branches dieselbe Stelle unterschiedlich ändern. Genau das wird jetzt absichtlich geübt.

Branch öffnungszeiten

Erstelle einen Branch öffnungszeiten und wechsle dorthin

- 1.
- 2.

Ergänze die Öffnungszeiten wie folgt:

```
<p>Öffnungszeiten: Montag bis Freitag 8:00 bis 18:00 Uhr</p>
```

Speicher die Änderung im Repository

```
git add index.html
git commit -m "Öffnungszeiten hinzugefügt"
```

Branch kontaktinfo

Erstelle einen Branch kontaktinfo und wechsle dorthin

```
git switch main
git switch -c kontaktinfo
```

Ergänze die Öffnungszeiten wie folgt:

```
<p>Öffnungszeiten: Dienstag bis Sonntag 9:00 bis 17:00 Uhr</p>
```

Speicher die Änderung im Repository

- 1.
- 2.

Merge zuerst den einen, dann den anderen Branch in main.

- 1.
- 2.
- 3.
- 4.

10. Konflikt verstehen und lösen

Öffne die index.html. Git markiert die widersprüchlichen Stellen mit Konfliktmarkierungen.

```
<====<<<<<< HEAD
<p>Öffnungszeiten: Montag bis Freitag 8:00 bis 18:00 Uhr</p>
=====
<p>Öffnungszeiten: Dienstag bis Sonntag 9:00 bis 17:00 Uhr</p>
>>>>>> kontaktinfo
```

Entscheide dich für eine Endfassung, entferne die Markierungen und schließe den Merge ab.

- 1.
2. _____ -m "Merge-Konflikt bei Öffnungszeiten gelöst"
3. git log --oneline --graph --all

Wichtig: Ein Konflikt ist kein Fehler von Git. Git signalisiert nur, dass die Entscheidung jetzt vom Menschen getroffen werden muss.

11. Merge abbrechen und Branch erzwingen löschen

In der Praxis kommt es vor, dass ein Merge doch nicht durchgeführt werden soll oder ein Branch verworfen wird.

```
git merge --abort
```

Erstelle außerdem einen kurzen “Experiment”-Branch, committe eine Änderung und versuche ihn ohne Merge zu löschen.

```
1.                                // wechsle in “experiment”-Branch
2. # kleine Änderung an script.js
3.                                // füge script.js der Staging Area hinzu
4.                                // Speichere die Version dauerhaft im Repository mit der
   Nachricht "Experimentelle Änderung"
5.                                // wechsele in den “Main” - Branch
6.                                // Lösche den Branch “experiment” nur dann, wenn Git erkennt,
   dass dessen Änderungen bereits sicher enthalten sind
7.                                // Lösche den Branch “experiment” erzwingend
```

Selbstkontrolle

- ☐ Wann ist `git merge --abort` sinnvoll?
- ☐ Warum warnt Git bei `git branch -d experiment`?
- ☐ Warum sollte `git branch -D` nur bewusst eingesetzt werden?

12. Abschluss und Übersicht

Nutze zum Abschluss die wichtigsten Befehle noch einmal gesammelt und bearbeite anschließend das Mini-Praxiszenario eigenständig.

Wichtige Befehle

Befehl	Bedeutung
<code>git branch</code>	vorhandene Branches anzeigen
<code>git branch <name></code>	neuen Branch anlegen
<code>git switch <name></code>	in einen Branch wechseln
<code>git switch -c <name></code>	Branch anlegen und sofort wechseln
<code>git merge <name></code>	einen Branch in den aktuellen Branch einfügen
<code>git merge --abort</code>	einen begonnenen Merge abbrechen
<code>git branch -d <name></code>	gemergten Branch löschen
<code>git branch -D <name></code>	Branch erzwungen löschen
<code>git log --oneline --graph --all</code>	Historie grafisch lesen

Mini-Praxisszenario: Erstelle einen Branch `angebote` mit einem neuen Angebotsfeld, einen Branch `farben` mit geändertem Farbschema, merge beide in `main`, lösche danach beide Branches und dokumentiere den Verlauf mit `git log --oneline --graph --all`.

Selbstkontrolle

- ☐ Warum arbeitet man neue Funktionen nicht direkt in `main` ein?
- ☐ Woran erkennst du einen Fast-Forward-Merge?
- ☐ Wie entsteht ein Merge-Konflikt?
- ☐ Wie hilft `git log --oneline --graph --all` beim Verstehen des Ablaufs?